

Task 1: Create a Dynamic Book Display using DOM Manipulation in JavaScript

You are tasked with building a simple interactive web page that allows users to add books to a list. The page must include input fields for the book title and author, a button to add the book, and a container that displays all added books. Each book entry should show the title, author, and a delete button to remove that specific entry. This exercise will help you practice essential DOM operations: selecting elements, creating new elements, appending them, handling events, and removing elements dynamically.

Required Methods

- `addBook()`
- `deleteBook(bookElement)`

Task Requirements

1. Create an HTML file with a basic structure that includes a heading (`<h1>My Book List</h1>`), two input fields – one for title (`id="titleInput"`) and one for author (`id="authorInput"`) – a button with the text "Add Book" (`id="addBtn"`), and a container div (`<div id="bookContainer"></div>`) where book cards will be displayed.
2. Use JavaScript to select the input fields, the button, and the container using either `getElementById` or `querySelector`.
3. Define a function named `addBook()` that will be invoked when the "Add Book" button is clicked (use `addEventListener`).
4. Inside `addBook()`, retrieve the current values from the title and author input fields.
5. Validate the inputs – if either field is empty, display an alert to the user and prevent the addition.
6. Create a new div element to serve as a book card and give it a class (e.g., `class="book-card"`) for styling purposes.
7. Set the content of the book card div to display the title and author (e.g., "Title: [title] | Author: [author]") using `innerHTML` or text nodes.
8. Create a "Delete" button inside the book card, set its text to "Delete", and assign it a class (e.g., `class="delete-btn"`).
9. Add an event listener to the "Delete" button so that clicking it removes the entire book card (its parent div) from the container – you may use a separate `deleteBook()` function or an anonymous function that directly removes the parent element.

10. Append the newly created book card to the bookContainer using appendChild.
11. Clear the input fields after successfully adding the book (set their values to an empty string).
12. (Optional) Display a counter showing the total number of books added, updating it each time you add or delete a book.
13. Ensure the delete functionality works for every book – including those added dynamically – by attaching listeners correctly (you may use event delegation or attach listeners to each button individually).
14. Add short comments in your JavaScript code explaining each major step (e.g., selecting elements, creating a book card, attaching events).
15. Make sure the code is clear, readable, and properly formatted – use consistent indentation and meaningful variable/function names.

Task 2: Create a Dynamic To-Do List with Filtering using DOM Manipulation in JavaScript

You are building an interactive to-do list application. The page must allow users to add tasks, mark them as complete/incomplete, delete individual tasks, and filter the list to show all tasks, only active tasks, or only completed tasks. This exercise will help you practice event delegation, class toggling, dynamic filtering, and managing state in the DOM.

Required Methods

- addTask()
- toggleTaskStatus(taskElement)
- deleteTask(taskElement)
- filterTasks(filterType)

Task Requirements

1. Create an HTML file with a basic structure that includes a heading (`<h1>My To-Do List</h1>`), an input field for the task description (`id="taskInput"`), a button with the text "Add Task" (`id="addBtn"`), a container div for the task list (`<div id="taskContainer"></div>`), and a filter section with three buttons: "All" (`id="filterAll"`), "Active" (`id="filterActive"`), and "Completed" (`id="filterCompleted"`).
2. Use JavaScript to select the input field, the add button, the task container, and all three filter buttons using `getElementById` or `querySelector`.
3. Define a function named `addTask()` that will be invoked when the "Add Task" button is clicked (use `addEventListener`).
4. Inside `addTask()`, retrieve the current value from the task input field.
5. Validate the input – if the field is empty (or contains only whitespace), display an alert to the user and prevent the addition.
6. Create a new div element to serve as a task item and give it a class (e.g., `class="task-item"`).
7. Set the content of the task item div to display the task description.
8. Create a "Complete/Incomplete" toggle button inside the task item with the text "✓" (or "Toggle"), and assign it a class (e.g., `class="toggle-btn"`). Clicking this button should call `toggleTaskStatus()` and toggle a CSS class (e.g., `class="completed"`) on the task item.

9. Create a "Delete" button inside the task item with the text "X" (or "Delete"), and assign it a class (e.g., class="delete-btn"). Clicking this button should call deleteTask() and remove the entire task item from the container.
10. Attach the toggle and delete event listeners **using event delegation** on the taskContainer – this ensures that dynamically added tasks also respond to clicks (do not attach listeners individually to each button).
11. Append the newly created task item to the taskContainer using appendChild.
12. Clear the input field after successfully adding the task (set its value to an empty string).
13. Define a function named toggleTaskStatus(taskElement) that accepts a task element, toggles a class (e.g., "completed") on it, and updates any visual styling (e.g., strikethrough text or color change) via CSS.
14. Define a function named deleteTask(taskElement) that accepts a task element and removes it from the DOM (using remove() or removeChild).
15. Define a function named filterTasks(filterType) that accepts a string ("all", "active", or "completed"). It should iterate over all task items in the container and show/hide them based on their completed status.
 - "all" – show all tasks.
 - "active" – show only tasks **without** the "completed" class.
 - "completed" – show only tasks **with** the "completed" class.
16. Attach click event listeners to the three filter buttons, each calling filterTasks() with the appropriate filter type.
17. (Optional) Display a task counter that shows the total number of tasks and the number of active/completed tasks, updating it every time a task is added, toggled, or deleted.
18. Add short comments in your JavaScript code explaining each major step (e.g., selecting elements, creating a task item, delegating events, filtering logic).
19. Make sure the code is clear, readable, and properly formatted – use consistent indentation and meaningful variable/function names.

Task 3: Create a Dynamic Shopping Cart with Price Calculation using DOM Manipulation in JavaScript

You are building an interactive shopping cart page. Users can add products by entering a product name, price, and quantity. Each item appears in a cart list with the ability to adjust the quantity (increase/decrease) or remove the item entirely. The page must automatically update the total price whenever items are added, removed, or quantities change. This exercise will help you practice DOM traversal, event delegation, numeric input handling, and real-time calculations.

Required Methods

- addItem()
- removeItem(itemElement)
- updateQuantity(itemElement, change)
- calculateTotal()

Task Requirements

1. Create an HTML file with a basic structure that includes a heading (`<h1>Shopping Cart</h1>`), an input field for the product name (`id="nameInput"`), an input field for the price (`id="priceInput"`), an input field for the quantity (`id="quantityInput"`), a button with the text "Add Item" (`id="addBtn"`), a container div for the cart items (`<div id="cartContainer"></div>`), and a separate div or `<p>` element to display the total price (`id="totalDisplay"`).
2. Use JavaScript to select the three input fields, the add button, the cart container, and the total display element using `getElementById` or `querySelector`.
3. Define a function named `addItem()` that will be invoked when the "Add Item" button is clicked (use `addEventListener`).
4. Inside `addItem()`, retrieve the current values from the name, price, and quantity input fields.
5. Validate the inputs – if the name is empty, price is not a positive number, or quantity is not a positive integer, display an alert to the user and prevent the addition.
6. Create a new div element to serve as a cart row and give it a class (e.g., `class="cart-item"`).
7. Inside the cart row, create elements to display the product name, the price, the current quantity, and two small buttons: a "+" button to increase the quantity and a

"-" button to decrease the quantity. Also add a "Remove" button (with text "X" or "Remove").

8. Set the initial quantity display to the value entered by the user.
9. Attach event listeners for the "+", "-", and "Remove" buttons **using event delegation** on the cartContainer – this ensures that dynamically added items also respond to clicks without needing to attach listeners individually.
10. When the "+" button is clicked, call `updateQuantity(itemElement, 1)` to increase the quantity by 1.
11. When the "-" button is clicked, call `updateQuantity(itemElement, -1)` to decrease the quantity by 1. If the quantity would become 0 (or less), automatically remove the item by calling `removeItem(itemElement)`.
12. When the "Remove" button is clicked, call `removeItem(itemElement)` to delete the entire cart row from the container.
13. Define the `updateQuantity(itemElement, change)` function – it should accept a cart row element and a numeric change (e.g., +1 or -1). It should find the quantity display element inside that row, update its text content to the new value, and then call `calculateTotal()`.
14. Define the `removeItem(itemElement)` function – it should accept a cart row element and remove it from the DOM (using `remove()` or `removeChild()`), then call `calculateTotal()`.
15. Define the `calculateTotal()` function – it should iterate over all existing cart rows (using `querySelectorAll()`), extract the price and current quantity from each row, compute the sum of (price × quantity) for every row, and update the `totalDisplay` element with the final total (formatted to two decimal places).
16. After successfully adding an item, clear the name, price, and quantity input fields (set their values to an empty string or default number).
17. Ensure that the total price updates immediately after adding an item, after removing an item, and after any quantity change.
18. (Optional) Add a "Clear Cart" button that removes all items at once and resets the total to \$0.00.

19. Add short comments in your JavaScript code explaining each major step (e.g., selecting elements, validating inputs, creating a cart row, delegating events, updating totals).
20. Make sure the code is clear, readable, and properly formatted – use consistent indentation and meaningful variable/function names.

Task 4: Create a Dynamic Quiz App with Scoring using DOM Manipulation in JavaScript

You are building an interactive quiz application. The page must display a set of multiple-choice questions loaded from a JavaScript array. Users can select one option per question, submit the quiz to see their score, and reset the quiz to retake it. Each question should display the question text and a list of clickable answer choices. When an answer is selected, the chosen option should be visually highlighted. After submission, the app must show the total score (number of correct answers) and clearly indicate which questions were answered correctly or incorrectly. This exercise will help you practice dynamic content generation, state management using data attributes, event delegation with dynamic elements, and conditional rendering.

Required Methods

- loadQuestions()
- selectAnswer(questionIndex, optionIndex)
- submitQuiz()
- resetQuiz()

Task Requirements

1. Create an HTML file with a basic structure that includes a heading (`<h1>JavaScript Quiz</h1>`), a container div for the questions (`<div id="quizContainer"></div>`), a "Submit Quiz" button (`id="submitBtn"`), a "Reset Quiz" button (`id="resetBtn"`), and a separate div or `<p>` element to display the score result (`id="resultDisplay"`).
2. In JavaScript, define an array of question objects – each object should have a question (string) and an options array (array of strings) and a correctAnswer (number representing the index of the correct option in the options array). Use at least 5 questions.
3. Use JavaScript to select the quiz container, the submit button, the reset button, and the result display element using `getElementById` or `querySelector`.

4. Define a function named `loadQuestions()` that dynamically generates the quiz content. It should iterate over the questions array and, for each question, create a new div element to serve as a question card with a class (e.g., `class="question-card"`).
5. Inside each question card, display the question text (e.g., using an `<h3>` or `<p>` element).
6. For each question's options array, create a set of clickable answer elements. You can use div elements, button elements, or label + input (radio) elements, but ensure that clicking an option visually highlights it. Give each option a class (e.g., `class="option"`) and a custom data attribute (e.g., `data-option-index`).
7. Attach click event listeners for all options **using event delegation** on the quizContainer – check if the clicked element has the option class, then retrieve the question index (store it as a `data-question-index` attribute on each question card) and the selected option index from the clicked element's data attribute, and call `selectAnswer(questionIndex, optionIndex)`.
8. Define the `selectAnswer(questionIndex, optionIndex)` function – it should accept a question index and an option index. It should remove the selected class from all options belonging to that question, then add the selected class to the specific option that was clicked, visually indicating the user's choice.
9. Ensure that each question card stores its index in a data attribute (e.g., `data-question-index`) for easy retrieval during event delegation.
10. Call `loadQuestions()` when the page loads so that the quiz renders immediately.
11. Define a function named `submitQuiz()` that will be invoked when the "Submit Quiz" button is clicked (use `addEventListener`).
12. Inside `submitQuiz()`, iterate over all question cards (using `querySelectorAll`), and for each card, find the selected option (the one with the selected class). If no option is selected for a question, mark that question as unanswered (you may display a warning).
13. For each answered question, compare the selected option index with the `correctAnswer` index from the questions array. Keep a counter of correct answers.
14. Display the final score in the `resultDisplay` element (e.g., "You scored 4 out of 5!").

15. Visually mark each question as correct (e.g., green border) or incorrect (e.g., red border) after submission, and also highlight the correct answer (e.g., green background) so the user can learn from their mistakes.
16. Disable further option clicks after submission (or ignore them) so the user cannot change answers once graded.
17. Define a function named `resetQuiz()` that will be invoked when the "Reset Quiz" button is clicked (use `addEventListener`).
18. Inside `resetQuiz()`, clear the `resultDisplay`, remove all visual feedback (borders, highlights), clear the selected class from all options, and re-enable the options so the user can take the quiz again – you may simply call `loadQuestions()` again to rebuild the quiz from scratch, or reset the existing DOM elements.
19. Add short comments in your JavaScript code explaining each major step (e.g., defining questions, rendering the quiz, handling option selection, grading logic, resetting the state).
20. Make sure the code is clear, readable, and properly formatted – use consistent indentation and meaningful variable/function names.

Task 5: Create an Interactive Notes App with Persistent Storage using DOM Manipulation in JavaScript

You are building a simple yet powerful notes application. Users can add notes with a title and content. Each note appears as a card in a grid layout. Users can delete individual notes and search through their notes by title or content in real time. All notes are automatically saved to the browser's `localStorage`, so they persist even after the page is refreshed or the browser is closed and reopened. This exercise will help you practice working with arrays of objects, generating unique IDs, storing and retrieving data from `localStorage`, implementing a live search filter, and using event delegation for dynamically created elements.

Required Methods

- `addNote()`
- `deleteNote(notelId)`
- `searchNotes(query)`

- saveNotes()
- loadNotes()

Task Requirements

1. Create an HTML file with a basic structure that includes a heading (`<h1>My Notes</h1>`), a form section containing an input field for the note title (`id="titleInput"`), a textarea for the note content (`id="contentInput"`), and a button with the text "Add Note" (`id="addBtn"`). Also include a search input field (`id="searchInput"`) with a placeholder like "Search notes...", and a container div for the notes grid (`<div id="notesContainer"></div>`).
2. In JavaScript, declare an empty array named `notes` that will hold all note objects. Each note object should have three properties: `id` (a unique number), `title` (a string), and `content` (a string).
3. Use JavaScript to select the title input, content textarea, add button, search input, and notes container using `getElementById` or `querySelector`.
4. Define a function named `addNote()` that will be invoked when the "Add Note" button is clicked (use `addEventListener`).
5. Inside `addNote()`, retrieve the current values from the title and content fields.
6. Validate the inputs – if the title is empty (or contains only whitespace), display an alert to the user and prevent the addition. (The content may be optional, but at least the title is required.)
7. Create a new note object with a unique `id` (use `Date.now()` or a counter), the provided title, and content. Push this object into the `notes` array.
8. Call the `saveNotes()` function to persist the updated array to `localStorage`, then call `renderNotes()` to refresh the display.
9. Clear the title and content fields after successfully adding the note (set their values to an empty string).
10. Define a function named `renderNotes()` that dynamically generates the note cards. It should clear the `notesContainer` and then iterate over the `notes` array (or a filtered subset, as described later). For each note, create a new `div` element with a class (e.g., `class="note-card"`) and store the note's `id` as a data attribute (e.g., `data-note-id`).

11. Inside each note card, display the note's title (using an `<h3>` element) and its content (using a `<p>` element). Also create a "Delete" button with the text "X" or "Delete", and assign it a class (e.g., `class="delete-btn"`).
12. Append each complete note card to the `notesContainer`.
13. Attach event listeners for the "Delete" buttons **using event delegation** on the `notesContainer` – check if the clicked element has the `delete-btn` class, then retrieve the `data-note-id` from the parent note card, and call `deleteNote(notelId)` with that ID.
14. Define the `deleteNote(notelId)` function – it should accept a note ID, filter the `notes` array to remove the note with that ID (using `filter`), then call `saveNotes()` and `renderNotes()` to update the display and storage.
15. Define the `saveNotes()` function – it should convert the `notes` array to a JSON string using `JSON.stringify()` and store it in `localStorage` under a key like `"notesData"`.
16. Define the `loadNotes()` function – it should retrieve the JSON string from `localStorage` using the same key, parse it back into an array using `JSON.parse()`, and assign it to the `notes` variable. If no data exists in `localStorage`, initialize `notes` as an empty array. After loading, call `renderNotes()` to display the notes.
17. Call `loadNotes()` when the page first loads so that any previously saved notes appear automatically.
18. Define a function named `searchNotes(query)` that will be invoked whenever the user types in the search input field (use the input event). Inside this function, get the current search query, convert it to lowercase, and filter the `notes` array to return only notes whose title or content contains the query string (case-insensitive). Then, instead of rendering all notes, render only the filtered results (you may pass the filtered array to a modified `renderNotes()` that accepts an optional array parameter, or handle the filtering inside `renderNotes()` itself). If the search field is empty, display all notes.
19. Attach an input event listener to the search field that calls `searchNotes()` with the current value of the search input.
20. Ensure that the search functionality works in real time – as the user types, the list of displayed notes updates immediately to show only matching notes.
21. (Optional) Add a small counter that displays the total number of notes and updates whenever a note is added, deleted, or the search filter changes.

22. Add short comments in your JavaScript code explaining each major step (e.g., defining the notes array, adding a note, saving to localStorage, rendering notes, delegating delete events, implementing search).
23. Make sure the code is clear, readable, and properly formatted – use consistent indentation and meaningful variable/function names.

Task 6: Create a Dynamic Tabbed Content Manager using DOM Manipulation in JavaScript

You are building an interactive dashboard application that displays different content sections organized into tabs. Users can add new tabs with custom content, switch between tabs to view different content, rename existing tabs, and delete tabs they no longer need. Each tab's content area should show rich HTML content (not just plain text). All tab data (names and content) must be stored in localStorage, so the state persists after page refresh. The application should automatically select the first tab when loaded and gracefully handle cases where no tabs exist. This exercise will help you practice managing complex state, implementing tab switching logic, handling inline editing with contenteditable, and using localStorage for persistent storage of structured data.

Required Methods

- addTab()
- switchTab(tabId)
- renameTab(tabId, newName)
- deleteTab(tabId)
- saveTabs()
- loadTabs()

Task Requirements

1. Create an HTML file with a basic structure that includes a heading (<h1>Tabbed Content Manager</h1>), a tab bar container (<div id="tabBar"></div>), a content display area (<div id="contentArea"></div>), an "Add Tab" button (id="addTabBtn"), and optionally a small status or count display showing the number of tabs.
2. In JavaScript, declare an empty array named tabs that will hold all tab objects. Each tab object should have three properties: id (a unique number), name (a string, e.g., "Tab 1"), and content (a string containing HTML, e.g., <p>This is tab 1 content</p>).

3. Use JavaScript to select the tab bar, content area, add tab button, and any other necessary elements using `getElementById` or `querySelector`.
4. Define a function named `addTab()` that will be invoked when the "Add Tab" button is clicked (use `addEventListener`).
5. Inside `addTab()`, create a new tab object with a unique id (use `Date.now()` or an incrementing counter), a default name like "Tab X" (where X is the number of tabs + 1), and default content such as `<p>Content for Tab X</p>`. Push this object into the `tabs` array.
6. Call `saveTabs()` to persist the updated array to `localStorage`, then call `renderTabs()` to refresh the tab bar and content area.
7. Automatically switch to the newly created tab by calling `switchTab()` with the new tab's ID.
8. Define a function named `renderTabs()` that dynamically generates the tab bar and content area. It should clear both the `tabBar` and `contentArea` containers, then iterate over the `tabs` array. For each tab, create a `div` element to serve as a tab button with a class (e.g., `class="tab-btn"`) and store the tab's id as a data attribute (e.g., `data-tab-id`).
9. Inside each tab button, display the tab's name (using a `<span>` or text node). Also create a small "Rename" icon/button (e.g., Pen) and a "Delete" icon/button (e.g., X) inside the tab button. Give each of these buttons appropriate classes (e.g., `class="rename-btn"` and `class="delete-btn"`).
10. Attach click event listeners for the tab switching, renaming, and deleting actions **using event delegation** on the `tabBar` container – check the clicked element's class to determine which action to perform, then retrieve the `data-tab-id` from the parent tab button and call the appropriate function (`switchTab()`, `renameTab()`, or `deleteTab()`).
11. For the active tab, add a special class (e.g., `class="active"`) to its tab button to visually distinguish it from inactive tabs.
12. In the `contentArea`, display the content of the currently active tab. You can set the `innerHTML` of the content area to the tab's `content` property so that it renders any HTML tags.
13. Define the `switchTab(tabId)` function – it should accept a tab ID, find the tab with that ID in the `tabs` array, update the active tab's visual state by removing

the active class from all tab buttons and adding it to the selected tab's button, and update the contentArea to display the selected tab's content.

14. Define the `renameTab(tabId, newName)` function – it should accept a tab ID and a new name string. Find the tab object with that ID in the `tabs` array, update its name property, then call `saveTabs()` and `renderTabs()` to refresh the display. To implement renaming, you can either:
 - Use a `prompt()` dialog to ask the user for the new name, or
 - Make the tab name contenteditable by clicking on it (more advanced). For this task, using `prompt()` is acceptable.
15. When the rename button is clicked, call `renameTab()` with the tab ID and the new name obtained from the user (handle cancellation gracefully – if the user cancels or enters an empty string, do not update the name).
16. Define the `deleteTab(tabId)` function – it should accept a tab ID, filter the `tabs` array to remove the tab with that ID (using `filter`), then call `saveTabs()` and `renderTabs()`. After deletion, if the deleted tab was the active tab, automatically switch to the first remaining tab (if any); if no tabs remain, display a message like "No tabs available. Click 'Add Tab' to create one." in the `contentArea`.
17. Define the `saveTabs()` function – it should convert the `tabs` array to a JSON string using `JSON.stringify()` and store it in `localStorage` under a key like `"tabsData"`.
18. Define the `loadTabs()` function – it should retrieve the JSON string from `localStorage` using the same key, parse it back into an array using `JSON.parse()`, and assign it to the `tabs` variable. If no data exists in `localStorage`, initialize `tabs` with at least one default tab (e.g., a tab named "Home" with content `<p>Welcome to the dashboard!</p>`). After loading, call `renderTabs()` and automatically switch to the first tab (or the previously active tab if you store the active tab ID separately).
19. Call `loadTabs()` when the page first loads so that any previously saved tabs appear automatically.
20. Ensure that the content area displays rich HTML content correctly – test by adding content with links, lists, or bold/italic text.
21. (Optional) Add the ability to edit the content of a tab directly in the content area using `contenteditable`, and save the changes when the user clicks a "Save Content" button or when they focus out (blur event).

22. (Optional) Add a tab count display that updates whenever tabs are added, renamed, or deleted.
23. Add short comments in your JavaScript code explaining each major step (e.g., defining the tabs array, rendering tabs, delegating events, switching tabs, renaming, deleting, saving to localStorage, loading from localStorage).
24. Make sure the code is clear, readable, and properly formatted – use consistent indentation and meaningful variable/function names.